

Dynamic Attack Graphs: Incremental Re-computation with Differential Dataflow

Stefania-Cristina Mozacu; Emil C. Lupu

Executive summary

This study implements a proof-of-concept system for incremental recomputation of attack graphs using the Timely + Differential Dataflow Rust libraries. Input facts (vulnerabilities, network edges, firewalls, attacker positions, goals) are translated into keyed collections and derived facts are computed using joins, antijoins, and iterative fixed-point computation. The approach reduces update cost for local changes in shallow topologies (star) while worst-case chain-like dependencies remain costly.

Key components

- Data model: `VulnerabilityRecord`, `NetworkAccessRule`, `FirewallRuleRecord`, `AttackerStartingPos`, `AttackerTargetGoal`
- Derived facts: `EffectiveNetworkAccess`, `AttackerCodeExecution`, `AttackerOwnsMachine`, `AttackerGoalReached`
- Operators: `antijoin` (for denies), `iterate` (recursion), `distinct` (deduplication)

Benchmarks (summary)

Topologies: star, chain, random-cut on chain. Metrics: initial full recomputation time, incremental update time, and speedup. Representative outcome: star topology shows large speedups; chain shows marginal improvements for cuts near the head.

Reproducibility

Place the Graphviz images (`graphiinitial.png`, `graphffinal.png`) in the project root's working directory. Key commands:

```
cargo build --release
cargo run --release --bin attack-graph
dot -Tpng graph_initial.dot -o graph_initial.png
docker build -t attack-graph .
docker run --rm attack-graph
```

Next steps

Scaling to multiple workers, integrating real vulnerability scanner output, and applying the approach to real network topologies.

Location of code

See the repository root. Key files: `src/rules.rs`, `src/benchmarks.rs`, `Dockerfile`, `graphiinitial.dot`